



Ducks in a Row

Certificate Lifecycle Automation & Management for ADCS

Installation & Configuration Guide

Make ADCS work like Let's Encrypt for your internal network.

Version 0.10.0-beta.1
2026

Table of Contents

1. Quick Start	3
2. System Requirements	6
3. Installation	9
4. Configuration	10
5. ADCS Configuration	13
6. Testing Your Installation	15
7. Connecting ACME Clients	17
8. Alert Configuration	20
9. Troubleshooting	23
10. Uninstalling	25

1. Quick Start

Get Ducks in a Row from a fresh download to a working ACME proxy. Plan for about fifteen minutes the first time, most of it spent on Active Directory permissions.

Step 1: Install Prerequisites

If you install with **Ducks-in-a-Row-Setup.exe** (recommended), the ASP.NET Core 10.0 runtime is installed for you, offline, from the installer itself. If you install the bare MSI instead, first download and install the **ASP.NET Core 10.0 Hosting Bundle** from the official .NET download page. Either way, install the ADCS management tools and reboot the server if prompted.

```
# Bare MSI only: download and install the .NET 10 Hosting Bundle from:
# https://dotnet.microsoft.com/download/dotnet/10.0

# Then install ADCS Remote Administration Tools (elevated PowerShell):
Install-WindowsFeature RSAT-ADCS-Mgmt
```

Critical: The ADCS RSAT tools must be installed on the Ducks in a Row server even though it is not the CA server. Ducks in a Row communicates with ADCS through the ADCS COM classes, which are registered by the RSAT feature. Without them, CA connectivity fails with COM activation errors.

Step 2: Run the Installer

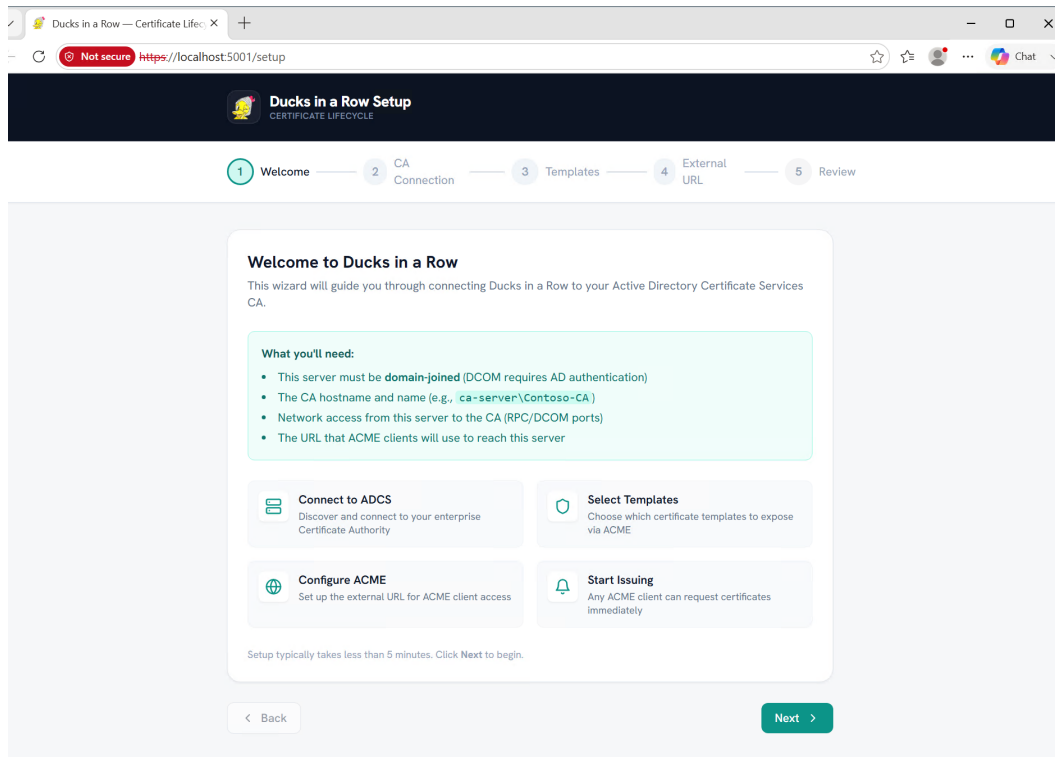
Run Ducks-in-a-Row-Setup.exe. It installs the ASP.NET Core runtime when the server does not have it, then opens the install wizard. On a server that already has the runtime, the bare MSI also works, either directly or from an elevated command prompt:

```
msiexec /i Ducks-in-a-Row.msi
```

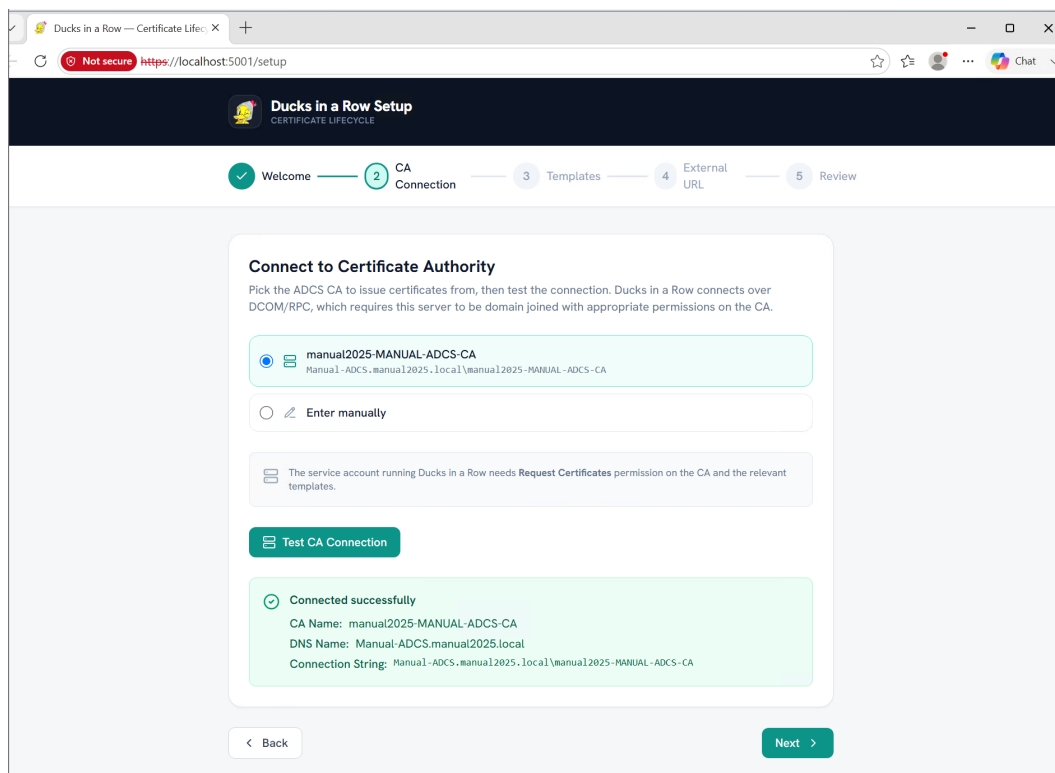
The install wizard asks for the destination folder, the data folder (database, logs, and runtime configuration), and whether to start the service immediately.

Step 3: Run the Setup Wizard

Open the setup page (the completion screen offers to open it for you) and sign in as a member of the administrator group. The wizard discovers the CAs published in Active Directory, tests the connection to the CA you pick, lists its templates and lets you choose one, validates the external URL, then applies the configuration and restarts the service by itself.



Welcome step: what you need before you start.



Connection step: pick a discovered CA and test it.

Ducks in a Row Setup
CERTIFICATE LIFECYCLE

Welcome — CA Connection — **3 Templates** — 4 External URL — 5 Review

Select a Certificate Template

Choose the ADCS template to expose via ACME. Only templates that can issue server authentication certificates are listed. The template gets its own ACME directory URL: `/acme/{template}/directory`

Web Server

Name: WebServer OID: 1.3.6.1.4.1.3.111.21.8.2637892.14261754.19613.8575474.3316996.56.1.16 `/acme/WebServer/directory`

ACME readiness

- Issues server authentication certificates
- No CA manager approval required
- No enrolment agent signatures required
- Subject and SAN are taken from the request
- Requires RSA keys, minimum 2048 bits
ACME clients must request a matching key type or the CA denies the request.

10 templates on this CA were hidden. Each either lacks the Server Authentication usage, has a subject built from Active Directory instead of the request, or could not be checked (its AD object could not be read, even though other templates on this CA resolved fine).

[< Back](#) [Next >](#)

Templates step: choose a template and check ACME readiness.

Ducks in a Row Setup
CERTIFICATE LIFECYCLE

Welcome — CA Connection — Templates — **4 External URL** — 5 Review

External URL

This is the base URL that ACME clients will use to reach the Ducks in a Row server. It must be reachable from every machine that needs to request certificates.

ACME clients will construct directory URLs like:

`https://manual-certus.manual2025.local:5001/acme/WebServer/directory`

Use HTTPS for production, and include the port. The service listens on 5001 for HTTPS (5000 for HTTP) instead of 443, because IIS and the ADCS web roles often occupy 443 on Windows servers and a port collision would stop the service from starting. The ports can be changed under Kestrel.Endpoints in appsettings.json; update the firewall rules to match.

Server URL

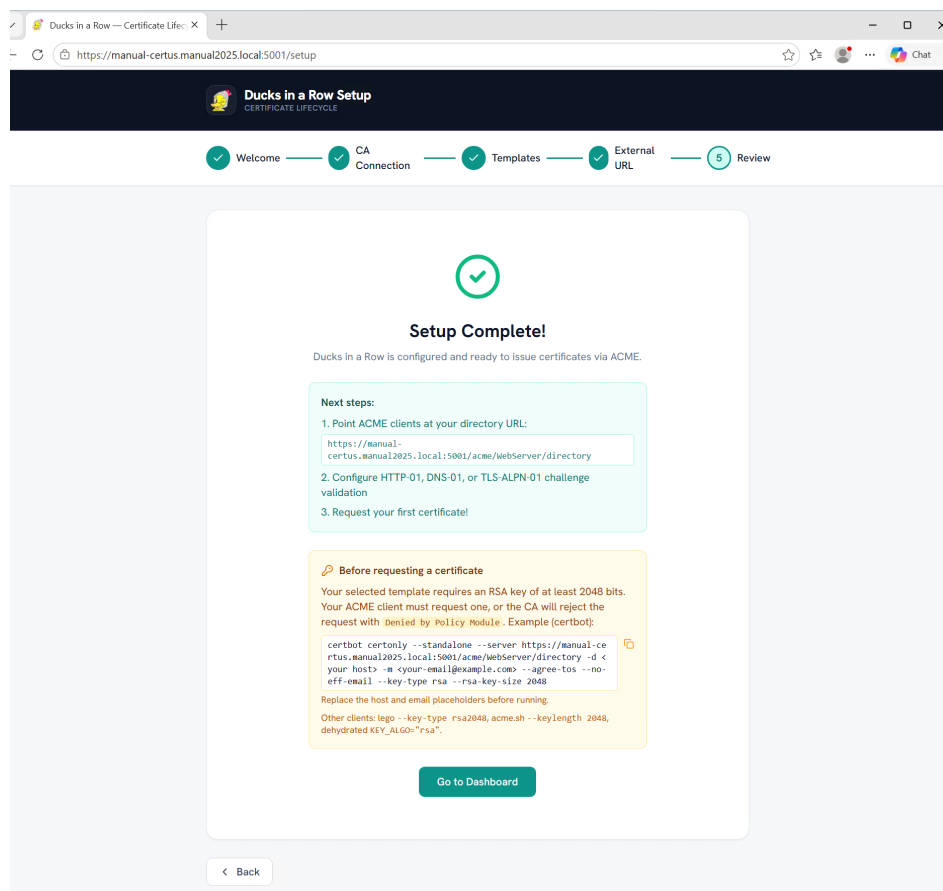
`https://manual-certus.manual2025.local:5001` [Validate](#)

Server reachable
The server answered at manual-certus.manual2025.local:5001. ACME clients should target: `https://manual-certus.manual2025.local:5001/acme/{template}/directory`

Certificate installed and in use
The service restarted and now serves the certificate issued by your CA.

[< Back](#) [Next >](#)

External URL step: confirm the URL and, optionally, enrol an HTTPS certificate.



Review step: apply the configuration and copy the certbot command.

Step 4: Verify

Open a browser and navigate to the following URLs:

```
https://your-server:5001/health (should return Healthy)
https://your-server:5001/setup (setup wizard)
https://your-server:5001 (dashboard)
```

The HTTPS endpoint uses a self signed certificate out of the box, so your browser warns on first visit. Plain HTTP on port 5000 is available for lab use.

Note: Until the setup wizard completes, the service runs unconfigured: the wizard and dashboard are reachable, and CA operations answer 503.

2. System Requirements

Server Requirements

Requirement	Details
Operating System	Windows Server. Verified on Server 2019 and Server 2025. On Server 2022, install current Windows updates first: the .NET 10 runtime requires Control-flow Enforcement Technology there, and an installation not serviced since early 2022 does not provide it (build 20348.587 is confirmed too old; check with winver). The service then crashes before it can log, and the installer reports the misleading Error 1920. Server 2019 predates the requirement; Server 2025 ships with it
Runtime	ASP.NET Core 10.0 (installed automatically by Ducks-in-a-Row-Setup.exe; install the Hosting Bundle manually only for the bare MSI)
Windows Feature	RSAT-ADCS-Mgmt (ADCS Remote Administration Tools)
Processor	2 x64 cores minimum. The work is mostly I/O bound (COM and RPC calls to the CA, plus SQLite), with short bursts of certificate crypto
Memory	1 GB minimum, 2 GB recommended
Disk	About 500 MB for the application and the .NET runtime. Plan at least 5 GB free for database growth, logs and upgrades. At roughly 1000 ACME accounts renewing over 3 years the database reaches the low hundreds of MB; log files are capped at 30 daily files
Network	TCP 5000 (HTTP) and TCP 5001 (HTTPS), both configurable
Domain	Must be domain-joined for ADCS connectivity

Installing Prerequisites

Run the following commands in an elevated PowerShell session on the Ducks in a Row server before installing:

```
# Install ADCS Remote Administration Tools
Install-WindowsFeature RSAT-ADCS-Mgmt

# Verify the feature is installed
Get-WindowsFeature RSAT-ADCS-Mgmt

# Verify the .NET 10 runtime is installed (after the setup bundle or
# a manual Hosting Bundle install)
dotnet --list-runtimes
```

Why RSAT-ADCS-Mgmt is required: Ducks in a Row communicates with the ADCS Certification Authority through the ADCS COM classes, which are registered on the system by the ADCS Remote Server Administration Tools. Without this feature installed, the COM objects cannot be instantiated and the service fails with `DISP_E_MEMBERNOTFOUND` (0x80020003) errors during certificate sync and CA queries.

ADCS Requirements

To issue certificates through ADCS, the following must be in place:

- An Active Directory Certificate Services (ADCS) Certification Authority (Enterprise CA)
- ADCS Remote Administration Tools (RSAT-ADCS-Mgmt) installed on the Ducks in a Row server
- DCOM/RPC network access from the Ducks in a Row server to the CA server (TCP 135 + dynamic ports 49152-65535)
- The server's machine account must have Enroll permission on the target certificate templates
- The server's machine account must have Read permission on the CA itself, or the dashboard inventory stays empty
- Certificate templates must be published to Active Directory

Network Ports

The following network ports must be open between the Ducks in a Row server and other systems:

Direction	Port	Protocol	Purpose
Inbound to the server	5000, 5001	TCP	ACME clients and dashboard browsers (HTTP and HTTPS)
Outbound to CA	135	TCP	DCOM/RPC endpoint mapper
Outbound to CA	49152-65535	TCP	DCOM/RPC dynamic ports
Outbound to targets	80	TCP	HTTP-01 challenge validation
Outbound to DNS	53	TCP/UDP	DNS-01 challenge validation
Outbound to targets	443	TCP	TLS-ALPN-01 challenge validation

3. Installation

What the Installer Does

The Ducks in a Row MSI installer performs the following actions automatically:

Action	Details
Install application files	C:\Program Files\Ducks in a Row\ (or the folder you choose)
Create the data folder	C:\ProgramData\Ducks in a Row\ (or the folder you choose; recorded in the registry)
Create logs directory	logs\ inside the data folder
Register Windows Service	DucksInARow (auto-start, runs as LocalSystem)
Start the service	Service starts immediately after install (optional)
Create firewall rules	Inbound TCP 5000 and 5001 (Ducks in a Row Certificate Proxy)
Add a Start Menu shortcut	Ducks in a Row shortcut that opens the web UI

Silent Installation

For automated deployments, use a silent install with logging. INSTALLFOLDER, DATAFOLDER, and START_SERVICE are optional properties:

```
msiexec /i Ducks-in-a-Row.msi /qn INSTALLFOLDER="D:\Ducks in a Row" DATAFOLDER="D:\DucksData" START_SERVICE=0 /l*v "C:\temp\ducks-install.log"
```

INSTALLFOLDER and DATAFOLDER default to the Program Files and ProgramData folders shown above. Choose the data folder at install time; moving it on a later upgrade is not supported. START_SERVICE=0 registers the service but leaves it stopped. A /qn install never launches a browser.

Service Recovery

The Ducks in a Row service is configured to restart automatically on failure. The recovery policy uses escalating delays:

Failure	Action	Delay
First failure	Restart the service	5 seconds
Second failure	Restart the service	10 seconds
Subsequent failures	Restart the service	30 seconds

4. Configuration

Configuration is layered. Shipped defaults live in `appsettings.json` in the installation folder. The setup wizard writes instance settings (the CA connection string and external URL) to `settings.json` in the data folder, which overrides the defaults and survives upgrades. Environment variables override both. After editing either file, restart the service for changes to take effect.

Core Settings

Setting	Default	Description
<code>CaConnectionString</code>	null	ADCS CA connection string in <code>hostname\CAName</code> format; the setup wizard sets it
<code>DatabasePath</code>	<code>ducks.db</code> in the data folder	Full path to the SQLite database file
<code>ExternalUrl</code>	null	Public URL that ACME clients use to reach the server; the setup wizard sets it
<code>SyncIntervalMinutes</code>	5	How often to sync certificates from the ADCS CA database
<code>RequestHistoryDays</code>	30	How far back to sync pending, denied, and failed requests so their detail pages can show the CA's own explanation. Issued and revoked certificates are always synced in full. Set to 0 to skip those passes
<code>EnableWalMode</code>	true	Enable SQLite WAL mode for better concurrent performance

Allowed Domains

The allowed domain list restricts which DNS names ACME clients can order certificates for. Each entry covers the domain itself and all of its subdomains: an entry of `home.local` also allows `web.home.local`, so wildcard entries are not needed and are not accepted. Orders for names outside the list are refused with a `rejectedIdentifier` error naming each refused domain, and every refusal is recorded on the dashboard activity feed.

The setup wizard turns the restriction on for new installs on a domain joined server, prefilled with the AD domain. Manage it later on the Settings page under Allowed Domains. The list is stored in `ducks-setup.json` in the data folder, not in `appsettings.json`, and unlike the settings above it applies immediately: no service restart is needed.

Scope: this is an ACME layer policy inside Ducks in a Row. The CA itself can still issue certificates for any name through its own tools (the Certification Authority console, `certreq`, or `autoenrollment`). To constrain the CA itself, use CA side

controls such as name constraints or template ACLs; the allowed domain list complements them, it does not replace them.

Authentication Settings

The dashboard and setup API use Windows Integrated Authentication and are limited to the administrator group.

Setting	Default	Description
Auth:Mode	Negotiate	Windows Integrated Authentication for the dashboard
Auth:AdminGroup	null	Group allowed into the dashboard and setup; null means the built in Administrators group
Auth:RequireHttps	true	HSTS and HTTP to HTTPS redirect outside development
Auth:TrustedProxies	[]	Reverse proxy IPs whose X-Forwarded-* headers are trusted

Minimum Required Configuration

The setup wizard writes everything the service needs. For reference, after setup the `settings.json` file in the data folder looks like this:

```
{
  "Certus": {
    "CaConnectionString": "CA-SERVER\\MyCA",
    "ExternalUrl": "https://ducks.yourdomain.local:5001"
  }
}
```

Important: The `ExternalUrl` must be reachable by ACME clients on your network. This URL is embedded in ACME directory responses and challenge URLs. Use HTTPS in production environments.

Kestrel Web Server

By default, Ducks in a Row listens on all interfaces on TCP 5000 (HTTP) and TCP 5001 (HTTPS). The HTTPS endpoint uses a self signed certificate out of the box, generated in the data folder as `ducks-selfsigned.pfx`, so browsers warn on first visit and ACME clients will not trust it.

The easiest fix is built in. On the external URL step, the setup wizard can enrol an HTTPS certificate for this server from the CA you connected, install it, and reload the service. The Settings page can renew it later. Use that first.

If you would rather supply your own certificate, point Kestrel at one your clients already trust (for example, a certificate issued by your own CA whose root your clients trust):

```
"Kestrel": {
  "Endpoints": {
    "Https": {
      "Url": "https://0.0.0.0:5001",
      "Certificate": {
        "Path": "C:\\ProgramData\\Ducks in a Row\\ducks.pfx",
        "Password": "your-pfx-password"
      }
    }
  }
}
```

For a quick lab test only, you can set `Auth:RequireHttps` to `false` and use plain HTTP on port 5000 instead.

Logging

Ducks in a Row uses Serilog for structured logging. Logs are written to `logs\` in the data folder with daily rotation and a 30-day retention policy. Adjust the minimum log level in the Serilog configuration section:

```
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  }
}
```

5. ADCS Configuration

Finding Your CA Connection String

The CA connection string follows the format `hostname\CAName`. The setup wizard finds it for you, so you rarely need to build it by hand. The manual options below are a fallback.

Option A: Let the Setup Wizard Discover It

Open `https://your-server:5001/setup` after installation. The wizard discovers the ADCS CAs published in Active Directory, lets you pick one, and tests connectivity through the same path the service uses. This is the recommended way and needs no connection string typed by hand.

Option B: From the CA Server

Open the Certification Authority MMC snap-in (`certsrv.msc`) on your CA server. The CA name appears in the console tree. The connection string is the CA server hostname followed by a backslash and the CA name.

Option C: Using PowerShell

Run this on any domain-joined machine to discover CAs in your forest:

```
certutil -config - -ping
```

This displays a list of available CAs. The output includes the connection string in the required format.

Certificate Template Permissions

For Ducks in a Row to issue certificates on behalf of ACME clients, the server's machine account needs Enroll permission on each certificate template you plan to use. To configure this:

1. Open the Certification Authority MMC (`certsrv.msc`) on your CA server.
2. Right-click the target certificate template and select Properties.
3. Go to the Security tab.
4. Add the Ducks in a Row server machine account (e.g., `DUCKS-SERVER$`).
5. Grant the Enroll permission. Do not grant Autoenroll.
6. Click OK and repeat for each template.

CA Read Permission

The machine account also needs Read permission on the CA itself (Certification Authority console, CA properties, Security tab). Read is what lets the dashboard sync the certificate inventory. Without it, enrolment still works but the dashboard stays empty.

Template URL Mapping

Each ADCS certificate template is exposed as its own ACME directory endpoint. ACME clients target the URL for the specific template they need:

Template Name	ACME Directory URL
WebServer	<code>https://your-server:5001/acme/WebServer/directory</code>
(your template name)	<code>https://your-server:5001/acme/{template-name}/directory</code>

Note: The template segment is the template's programmatic name (its AD cn) or its display name, matched case insensitively. Display names with spaces work; the client URL encodes them.

6. Testing Your Installation

Service Status

Verify the Ducks in a Row service is running:

```
Get-Service DucksInARow

# Expected output:
# Status Name DisplayName
# -----
# Running DucksInARow Ducks in a Row Certificate Proxy
```

Health Endpoint

The health check endpoint reports the status of the database and CA connectivity:

```
Invoke-RestMethod https://your-server:5001/health

# Returns: Healthy, Degraded, or Unhealthy
```

Status	Meaning
Healthy	Database is accessible and CA connection is working
Degraded	Database is working but CA is not reachable
Unhealthy	Database is not accessible

ACME Directory

Test that ACME endpoints are responding correctly:

```
Invoke-RestMethod https://your-server:5001/acme/WebServer/directory | ConvertTo-Json

# Expected: JSON with newNonce, newAccount, newOrder URLs
```

Firewall Verification

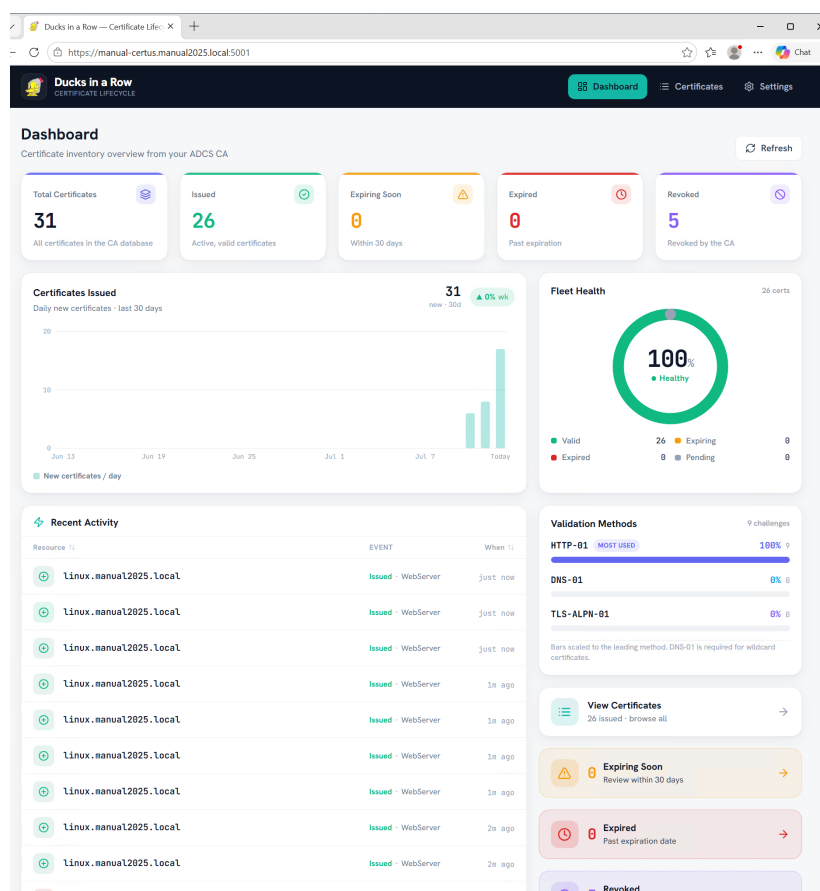
Confirm the firewall rules (HTTP and HTTPS) were created by the installer:

```
Get-NetFirewallRule -DisplayName "Ducks in a Row Certificate Proxy"
```

Dashboard Access

Open <https://your-server:5001> in a browser and sign in as a member of the administrator group. The dashboard displays certificate inventory, expiry status, and template information. On first access, you may be

redirected to the setup wizard at /setup.



The dashboard after the first sync, showing the certificate inventory and expiry tiles.

CA Connectivity Test

The setup wizard at <https://your-server:5001/setup> includes a built in connectivity test. Click **Test Connection** to verify DCOM/RPC communication with your ADCS CA. The test will report the CA name, server, and available templates.

7. Connecting ACME Clients

Ducks in a Row implements RFC 8555 (ACME), so any standard ACME client works out of the box. No agent or proprietary software is needed on your endpoints. Use the HTTPS endpoint (5001) in production; plain HTTP (5000) is for lab use only. The example commands are representative; exact flags vary by client version, so check each client's own documentation for your release.

Trusting the server TLS: ACME clients reject an untrusted TLS certificate on the ACME server itself. Give Ducks in a Row a certificate your clients already trust (for example, one chained to your internal CA root), or add that root to each client's trust store. The examples below note the per client switch for a lab where the server still uses its self signed certificate.

Certbot (Linux)

```
# HTTP-01, certbot answers the challenge itself on port 80
certbot certonly --standalone \
  --server https://your-server:5001/acme/WebServer/directory \
  --email you@example.com \
  -d host.corp.example.com \
  --key-type rsa --rsa-key-size 2048
```

Key type: If your ADCS template uses an RSA CSP (the default Web Server ACME template does), your ACME client must request an RSA key. Most modern clients default to elliptic curve keys, which the CA policy module rejects at finalize with Denied by Policy Module. The last line above forces RSA for certbot. Other clients: `lego --key-type rsa2048`, `acme.sh --keylength 2048`, `dehydrated KEY_ALGO="rsa"`.

Accounts do not need to be registered in advance and no external account binding is required; the client creates an account from its own key on first use. For a lab server with an untrusted TLS certificate, set `REQUESTS_CA_BUNDLE` to your CA root PEM so certbot trusts the endpoint.

win-acme (Windows)

Run `wacs.exe` and, in the menu, set the ACME server to the template directory URL, then create a certificate. Unattended example:

```
wacs.exe --source iis --siteid 1 ^
  --baseuri https://your-server:5001/acme/WebServer/directory
```

win-acme uses the Windows certificate trust store, so import your CA root on the machine running it if the server certificate is not already trusted.

Caddy

```
# Caddyfile
{
  acme_ca https://your-server:5001/acme/WebServer/directory
  # For an internal CA, point Caddy at the root so it trusts the ACME endpoint
  acme_ca_root /etc/ssl/certs/corp-root.pem
```

```
    email you@example.com
  }

  host.corp.example.com {
    respond "Hello from Caddy"
  }
```

Traefik

```
# traefik.yml
certificatesResolvers:
  ducks:
    acme:
      caServer: https://your-server:5001/acme/WebServer/directory
      email: you@example.com
      storage: /etc/traefik/acme.json
      httpChallenge:
        entryPoint: web
```

Traefik uses the host or container trust store. Add your CA root there if the ACME endpoint certificate is not already trusted.

Posh-ACME (PowerShell)

```
Import-Module Posh-ACME

# Point Posh-ACME at the template directory.
# Add -SkipCertificateCheck only for a lab server with untrusted TLS.
Set-PAServer -DirectoryUrl https://your-server:5001/acme/WebServer/directory

New-PAAccount -Contact you@example.com -AcceptTOS

New-PACertificate -Domain host.corp.example.com -Plugin WebRoot `
  -PluginArgs @{ WebRootPath = 'C:\inetpub\wwwroot' }
```

Challenge Types

Type	Use Case	How It Works
HTTP-01	Standard web servers	Fetches http://{domain}/.well-known/acme-challenge/{token}
DNS-01	Wildcards, hosts with no inbound HTTP	Looks up the TXT record at _acme-challenge.{domain}
TLS-ALPN-01	TLS only environments	Opens a TLS connection on port 443 and checks the ALPN certificate

The Ducks in a Row server is the party that performs validation, so the server needs outbound reachability to the domain or DNS being validated.

Outbound validation is screened. The server always refuses to validate targets on loopback, link local (including the cloud metadata address), and IPv6 unique local addresses. The RFC 1918 private ranges are allowed by default, because an internal CA usually issues for exactly those addresses. Set `Certus:Acme:ChallengeValidation:BlockPrivateRanges` to `true` when every validation target is public, or list extra ranges in `AdditionalBlockedCidrs`. The hardening guide in the docs folder covers the tradeoff.

8. Alert Configuration

Ducks in a Row monitors certificate expiry and sends notifications through email and webhooks. Alerts are included in the free tier with no limits.

Alert Thresholds

By default, alerts fire at 30, 14, 7, and 1 day(s) before certificate expiry. Each certificate receives at most one alert per threshold (no duplicates). The monitor checks for expiring certificates every 60 minutes by default (`CheckIntervalMinutes`).

SMTP Email Alerts

The whole SMTP transport can be set from the Alerts card on the Settings page: relay host, port, transport security, username and password, sender address and sender name, and the recipients. That is the recommended route, it needs no file editing, and the password is stored encrypted in the data directory rather than in a configuration file. The block below is what a fresh install starts from and remains available for scripted or unattended configuration; the file remains the only way to configure the webhook. `TlsMode` accepts `none`, `starttls` or `implicit`; without it, `UseSsl` plus the port decide (465 means implicit TLS, anything else negotiates mandatory STARTTLS).

```
"Certus:Alerts": {
  "Enabled": true,
  "CheckIntervalMinutes": 60,
  "ThresholdDays": [30, 14, 7, 1],
  "Smtp": {
    "Host": "smtp.yourdomain.local",
    "Port": 587,
    "TlsMode": "starttls",
    "Username": "ducks@yourdomain.local",
    "Password": "your-smtp-password",
    "FromAddress": "ducks@yourdomain.local",
    "FromName": "Ducks in a Row",
    "Recipients": ["admin@yourdomain.local"]
  }
}
```

Once a setting has been saved from the dashboard, the file no longer decides it. Saving on the Alerts card writes the whole writable set, `Enabled`, `CheckIntervalMinutes`, `ThresholdDays`, and every SMTP transport field, into a settings file in the data directory, which outranks `appsettings.json`. Editing those keys in the file after that has no effect, and the Alerts card lists which ones it now owns so the situation is visible rather than puzzling. A password saved from the dashboard is protected with the machine's Data Protection keyring: it never appears in any file in readable form, it is never returned by the API, and after restoring the data directory onto a different machine it cannot be decrypted by design, so save it again from the Settings page there. The webhook block is never written by the dashboard and is always read from the file.

An environment variable or command line argument still outranks both. Where one is in use, the Alerts card disables that field and says why, rather than accepting an edit that could never come into force.

Webhook Alerts

Webhooks send a JSON payload via HTTP POST. The payload is signed with HMAC-SHA256 using the configured secret. The signature is sent in the X-Certus-Signature header.

```
"Webhook": {
  "Url": "https://hooks.slack.com/services/YOUR/WEBHOOK/URL",
  "Secret": "your-hmac-secret",
  "Headers": {
    "X-Custom-Header": "value"
  }
}
```

Alert History

Sent alerts are recorded and available through the API at GET /api/alerts/history and GET /api/alerts/summary, and in the dashboard on the Settings page.

Alerts in the Dashboard

The Alerts card on the Settings page shows whether expiry monitoring is on, how often it checks, the warning thresholds in force, who is on the email recipient list, whether a webhook is configured, and the most recent alerts including any that failed.

Expiry monitoring, the check interval, the warning thresholds, the SMTP relay host, the sender address and the recipient list can all be changed here and saved. **A saved change is not in force until the service restarts.** Alerting reads its settings once when the service starts, so the card keeps showing what the service is actually running on and offers a **Restart now** button beside the notice. Until that restart, the figures on the card and the expiry counts elsewhere in the dashboard reflect the old settings, which is deliberate: appearing to have changed monitoring when it has not is worse than asking for a restart.

The card never shows the SMTP username or password, the webhook secret, or the webhook address, and none of them can be set from it. Sending one is refused rather than ignored. The webhook address is withheld because it commonly carries an access token in the URL, which is how most webhook services authenticate. The relay host and sender address are shown, because neither is a credential and a field that can be edited but not seen cannot be edited safely.

The card also has a **Send a test** button, which delivers over every configured channel and reports what each one did. Use it after changing SMTP settings: mail configuration fails quietly, and this is the fastest way to find out that it has. The test is clearly marked as a test in both channels, so a recipient cannot mistake it for a real expiry warning: the email subject leads with [Ducks in a Row TEST], and the webhook payload carries the event test.alert with test: true and no certificate list. It is signed with the same HMAC secret as a real alert, so it proves the real delivery path.

A test writes nothing to the alert history, so it cannot be mistaken later for a real warning and cannot suppress one. There is a cooldown of about a minute between tests, shared by everyone signed in.

A delivered test proves delivery only. It does not mean expiry monitoring is switched on: that is the separate Enabled setting, and the card says so when it is off.

9. Troubleshooting

Service fails to start

Check the Windows Event Log (Application) and the log files at `logs\` in the data folder. Common causes: missing .NET 10 runtime, invalid `appsettings.json` or `settings.json` syntax (trailing commas, missing quotes), or ports 5000 or 5001 already in use by another process. To see the actual error, run the executable directly from an elevated PowerShell: & "C:\Program Files\Ducks in a Row\DucksInARow.Service.exe"

CA operations answer 503, or the dashboard routes to the setup wizard

No CA is configured yet. The service starts unconfigured and never falls back to a fake CA on its own. Complete the setup wizard; it discovers the CAs in Active Directory, tests the connection, applies the configuration, and restarts the service.

COM activation or `DISP_E_MEMBERNOTFOUND` errors during CA operations

The ADCS COM classes are not registered on the Ducks in a Row server. Install the ADCS Remote Administration Tools: `Install-WindowsFeature RSAT-ADCS-Mgmt` and then restart the service. This feature must be installed even though this server is not the CA server. The COM class registrations are provided by the RSAT feature package.

Health endpoint returns Degraded

The database is working but the CA is not reachable. Verify the `CaConnectionString` (in `settings.json` in the data folder) is correct and uses the format `hostname\CAName`. Confirm DCOM/RPC traffic is not blocked by a firewall (TCP 135 and dynamic ports 49152-65535), and the server has network access to the CA. Test with: `certutil -config "SERVER\CA" -ping`

Health endpoint returns Unhealthy

The SQLite database is not accessible. Verify the `DatabasePath` points to a valid, writable location. Check that the data folder exists and the `LocalSystem` account has write access.

ACME client gets connection refused

Verify the service is running (`Get-Service DucksInARow`), the firewall rule exists (`Get-NetFirewallRule -DisplayName "Ducks in a Row"`), and the `ExternalUrl` in configuration matches how clients reach the server.

Certificate request fails with template error

The template name in the directory URL must match a template the CA publishes, either by its programmatic name (AD cn) or its display name; names are matched case insensitively. Confirm the template is published to Active Directory and the server's machine account has Enroll permission.

ACME client reports a TLS or certificate trust error

The HTTPS endpoint uses a self signed certificate by default (`ducks-selfsigned.pfx` in the data folder), and ACME clients reject untrusted TLS on the ACME server. Give Ducks in a Row a certificate your clients trust, or add that certificate root to each client trust store.

Cannot reach the dashboard, or get a 401

The dashboard and setup API use Windows Integrated Authentication and are limited to the administrator group. Sign in as a member of the built in Administrators group, or set `Auth:AdminGroup` to the Windows or Active Directory group you want to allow, then restart the service.

DCOM authentication errors

The Ducks in a Row server must be domain joined. Verify the machine account has not been disabled or moved to a restricted OU. Check that DCOM port ranges (TCP 135 + dynamic ports 49152-65535) are open between this server and the CA server.

Service exits silently with no log output

Run the executable directly to see the error: `dotnet "C:\Program Files\Ducks in a Row\DucksInARow.Service.dll"` Common causes include malformed JSON in `appsettings.json` or `settings.json` (use a JSON validator to check syntax), missing .NET runtime, or file permission issues. Set the environment variable `ASPNETCORE_ENVIRONMENT=Development` before running to enable detailed error output.

Dashboard shows no certificates

The machine account needs Read permission on the CA (Certification Authority console, CA properties, Security tab); without it, enrolment still works but the inventory sync cannot read the CA database and the log reports "CA view access denied". The sync also runs only every `SyncIntervalMinutes` (default 5), so wait for the first cycle or restart the service to force one.

ACME order rejected: domain is not in the allowed domain list

The allowed domain restriction is on and the requested name falls outside every allowed domain. The client sees a `rejectedIdentifier` error naming the refused domains. On the Settings page under Allowed Domains, add the domain (subdomains of an entry are covered automatically) or turn the restriction off. Changes apply immediately with no restart, and each rejected order also appears on the dashboard activity feed as Rejected.

10. Uninstalling

Ducks in a Row can be removed through Windows Add/Remove Programs or from the command line:

```
msiexec /x Ducks-in-a-Row.msi
```

What the uninstaller removes:

- All application files from C:\Program Files\Ducks in a Row\
- The DucksInARow Windows Service registration (the service is stopped first)
- The Ducks in a Row Certificate Proxy firewall rules

What the uninstaller preserves:

- The SQLite database (ducks.db in the data folder)
- Log files (logs\ in the data folder)
- The data folder itself (default C:\ProgramData\Ducks in a Row\)

To perform a complete removal including all data, delete the data directory manually after uninstalling:

```
Remove-Item -Recurse -Force "C:\ProgramData\Ducks in a Row"
```